Expense Tracker App — Build Plan

Oct 19, 2026 · @Keshav

Reality check

The core idea is solid, but four parts of it as described will hurt you. Fix these before writing any code.

1. "An app on my phone" — don't build a native app. Publishing on the Play Store needs a paid developer account, and new personal accounts must run a closed test with a dozen-plus testers for two weeks. iPhone friends need Apple's paid developer program or TestFlight. *Instead:* build a **PWA** — a website that installs to the home screen, opens full-screen, works offline, and runs on Android and iPhone. It deploys free on Vercel, like your other projects.

2. "Attach an LLM to classify" — not for every entry. If you type "150 food", you already gave the category; there is nothing to classify. An LLM call per entry adds 0.5–2 s of delay, needs internet, burns quota, and can get it wrong. *Instead:* a local parser handles the common case instantly, and a keyword memory learns your habits ("zomato" → Food after one correction). The LLM is only a fallback for messy text. After two weeks of use, it should rarely be called.

3. "Give it to friends" — you'd be holding their financial data. One shared database means their spending sits on your server: breach risk, hosting cost, and they have to trust you. *Instead:* go **local-first**. Each person's data lives only on their own phone. No accounts, no database to run.

4. The real killer is you stopping, not missing features. Expense apps die from abandonment around day 10. With UPI you make many small payments a day, and manual logging is friction. *Instead:* design every screen for entry in under 5 seconds, add an evening reminder, and make gaps painless with a lump-sum "untracked" entry.

One honest trade-off: a PWA cannot read bank SMS, so there is no automatic import like Axio or Walnut-style apps. That is the price of zero permissions and full privacy. If auto-import ever becomes a must-have, that is the one reason to go native Android later.

What the app is

A home-screen app where you type "150 food", it's saved in under 5 seconds, and a summary is one tap away.

Design rules every feature must pass:

- **Type-first.** The home screen is one text box with the keyboard already open. Taps and menus are the backup, not the main path.
- **Instant and offline.** Saving never waits on the network. The LLM, when used, fixes the category after the entry is already saved.
- **Private by default.** Data stays on the phone. The only thing that ever leaves is the text of an entry the parser couldn't categorise.
- **Zero cost, zero setup.** Friends open a link and add it to the home screen. No sign-up, no password.
- **Always show what was understood.** After each entry, a chip shows "₹150 · Food · Today". One tap fixes any part of it.

Features

v1 is only what you need to log daily and see where money went; everything else waits until friends ask for it.

Feature	What it does	Phase
Quick-add box	One input on the home screen, auto-focused. Type and hit enter.	v1
Smart parsing	Reads amount, category, date words ("yesterday", "mon") and a note from free text.	v1
Confirmation chip	Shows "₹150 · Food · Today" after saving; tap any part to fix it.	v1
Keyword memory	Learns that "zomato" means Food after you pick or correct it once. Seeded with common Indian merchants.	v1
Custom categories	Add, rename, reorder, set an emoji, archive (history is kept). Defaults: Food, Groceries, Transport, Shopping, Bills, Care, Fun, Other.	v1

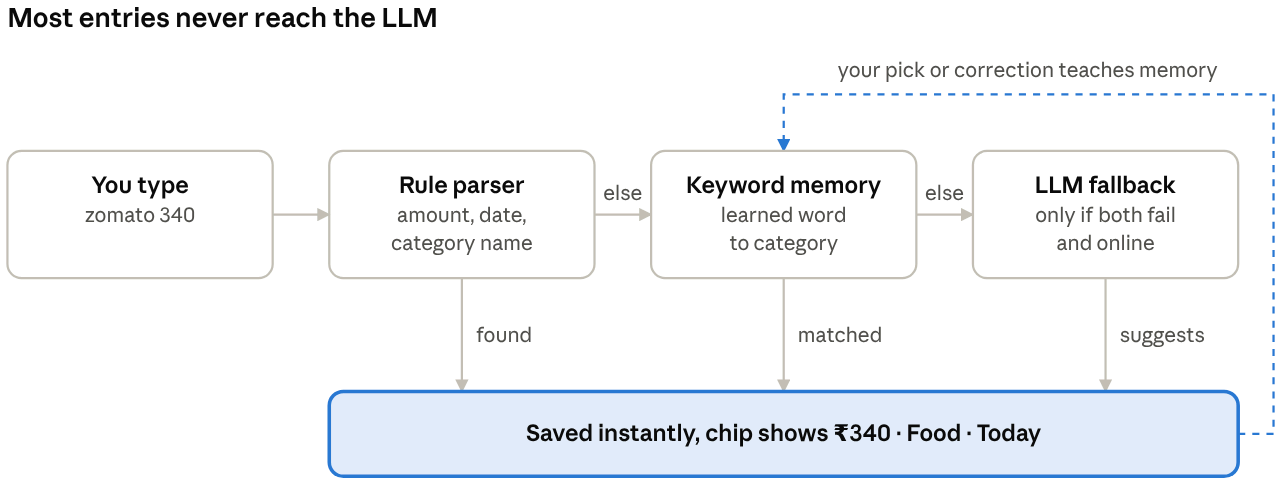
Feature	What it does	Phase
Entry list	Recent entries grouped by day. Tap to edit, swipe to delete, with an undo toast.	v1
Summary screen	Today / week / month totals and category breakdown (details below).	v1
Backup	Export and import as JSON (full backup) and CSV (for Excel). Reminder if no backup in 30 days.	v1
Installable + offline	Add to home screen; works with no internet.	v1
LLM fallback	Categorises messy entries the parser can't ("dinner w rohan + auto 340").	v1.5
Multi-entry line	"150 food, 40 auto" saves two entries.	v1.5
Category budgets	Monthly limit per category, warning at 80%.	v2
Evening reminder	One notification a day: "Anything to log today?"	v2
Recurring entries	Rent, subscriptions, auto-added monthly.	v2
Monthly insight	A short plain-English summary written by the LLM from numbers your code computed.	v2
Voice input	Speak "150 on food". Reliable on Android Chrome, patchy on iPhone.	v2
Share summary	Send a month's summary as text or image to WhatsApp.	v2

Explicitly **not** building:

- **Bank/SMS auto-import** — a PWA can't read SMS, and it's a heavy privacy cost.
- **Bill splitting** — Splitwise already does this well; don't compete with it.
- **Income, investments, net worth** — a different app. Stay an expense logger.
- **Accounts and cloud sync** — only if someone actually needs two devices or loses data.
- **Multi-currency** — rupees only.

How quick entry works

Three layers, cheapest first: a rule parser, then your keyword memory, then the LLM only if both fail. The amount always comes from code, never from the LLM.



entry flow · 3 layers, 1 learning loop

Every path ends with the entry already saved; your corrections feed keyword memory, so the next similar entry never needs the LLM.

Inputs the parser must handle:

You type	Amount	Category	Date	Note
150 food	₹150	Food	today	—
food 150	₹150	Food	today	—
₹150 on food	₹150	Food	today	—
1k shopping	₹1,000	Shopping	today	—
1.5k myntra	₹1,500	Shopping (memory)	today	myntra
zomato 340	₹340	Food (memory)	today	zomato
200 auto yesterday	₹200	Transport (memory)	yesterday	auto
150 fod lunch w team	₹150	Food (typo match)	today	lunch w team

You type	Amount	Category	Date	Note
150	₹150	ask: category chips	today	—
dinner w rohan + auto 340	₹340	LLM suggests	today	dinner w rohan + auto

The rule parser, step by step:

1. **Normalise.** Lowercase; strip ₹, rs, rs., commas, and filler words (on, for, spent, paid).
2. **Find the amount.** Regex for a number with optional decimals and a k suffix (1k, 1.5k). If there are several numbers, prefer one next to ₹/rs; otherwise take the last. No number → inline error, nothing saved.
3. **Find the date.** today, yesterday, day names (last occurrence), "2 days ago", dd/mm. Default: today.
4. **Find the category.** Check each remaining word against category names and aliases, then keyword memory, then a one-letter-typo match on category names.
5. **Keep the rest as the note.**
6. **No category found?** Save it as Uncategorised immediately. Online: ask the LLM and show its pick as a one-tap suggestion. Offline: queue it.

Keyword memory is a small local table: word → category, with a use count. Every saved entry teaches it its note words; every correction overwrites. Seed it on first launch with obvious ones: Swiggy/Zomato → Food, Blinkit/Zepto/BigBasket → Groceries, Uber/Ola/Rapido/metro/auto → Transport, Amazon/Myntra/Flipkart → Shopping. This is what makes the LLM unnecessary for most entries.

The LLM call, when it happens:

- Send only the leftover text plus your category list. Ask for JSON: category (must be one from the list) and an optional cleaned note.
- Temperature 0, a small fast model, short timeout (3 s). On timeout or junk output, leave it Uncategorised.
- Validate the reply in code. A category not on the list is rejected.
- Never let it change the amount or the date.

Summaries

The summary screen answers one question in a glance: where did my money go this month? All numbers are computed in code; the LLM never does arithmetic.

Top to bottom:

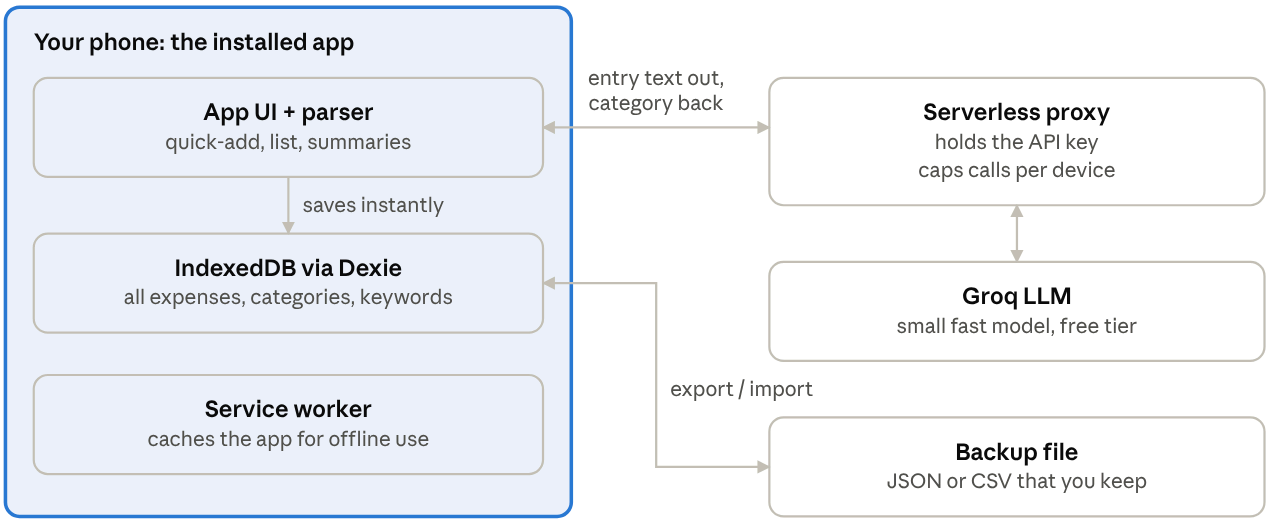
1. **Period switch** — Today · Week · Month · custom range. Month is the default.
2. **Total, with context** — "₹8,400 so far this month" plus a pace comparison: "vs ₹7,100 by this date last month". The comparison is what makes a number meaningful.
3. **By category** — horizontal bars, largest first, with ₹ and % share. Tap a category to see its entries.
4. **Day by day** — small bar chart of daily spend, so spikes stand out.
5. **Biggest 5 entries** — the few items that usually explain a bad month.
6. **Budgets** (v2) — progress bar per category with a limit set.
7. **Monthly insight** (v2) — 3–4 sentences written by the LLM from a stats object your code builds (totals, top categories, change vs last month). It describes; it doesn't calculate.

Skip a free-form "ask anything" chat box. Turning questions into correct queries is fragile, and tapping a category plus a date filter answers most of what you'd ask.

Tech stack and architecture

A React PWA that stores everything on the phone, plus one tiny serverless function that holds the LLM key. No database server, no backend to keep alive.

Your data stays on the phone; only stuck entries leave it



architecture · phone vs network

Vercel only serves the app's files; after install, the app runs from the phone, and the proxy is the single thing that touches the network.

Piece	Choice	Why
UI	React + Vite	You already ship with it.
Installable/offline	vite-plugin-pwa (service worker + manifest)	Adds install and offline caching with little config.
Storage	IndexedDB via Dexie.js	Real local database in the browser; versioned schema migrations.
Charts	Recharts or Chart.js	Only 2–3 simple charts needed.
Styling	Tailwind	Fast to build a clean mobile layout.
Hosting	Vercel	Free, auto-deploys from GitHub.
LLM proxy	Vercel serverless function or Cloudflare Worker	Keeps the API key off the phone; enforces usage caps.
LLM	A small fast model on Groq's free tier	Cheap, quick; check current limits in the Groq console.

Piece	Choice	Why
Tests	Vitest	Same ideas as pytest, for JavaScript.
CI	GitHub Actions running the parser tests on every push	A broken parser never reaches friends.

Three non-negotiables:

- **Never put the API key in the frontend.** Anyone can open the page source and drain your quota. The key lives only in the proxy's environment variables.
- **Store money as integer paise** (₹150 → 15000). Floats give errors like 0.1 + 0.2 ≠ 0.3 in totals.
- **Store the date as a local YYYY-MM-DD string**, separate from the timestamp. Otherwise entries near midnight land on the wrong day when converted from UTC.

Data model

Four IndexedDB tables. Keep the original typed text on every entry so you can re-parse history if the parser improves.

Table	Field	Type	Note
expenses	id	string (UUID)	Generated on the phone.
expenses	amountPaise	integer	₹150 → 15000.
expenses	categoryId	string	"uncategorised" until resolved.
expenses	note	string	Leftover words.
expenses	rawText	string	Exactly what was typed.
expenses	date	string YYYY-MM-DD	Local date; indexed for summaries.
expenses	createdAt / updatedAt	timestamp	For sorting and backups.
expenses	source	rule · memory · llm · manual	How the category was decided; tells you how often the LLM is needed.

Table	Field	Type	Note
expenses	deleted	boolean	Soft delete, so undo works.
categories	id, name, emoji	string	Name is unique, case-insensitive.
categories	aliases	string list	"food", "meals", "eating out".
categories	order, archived	number, boolean	Archived ones hide but keep history.
keywords	word	string (key)	Lowercased, e.g. "zomato".
keywords	categoryId, count, lastUsed	string, number, timestamp	Highest count wins on conflict.
settings	budgets, reminderTime, lastBackupAt, deviceId, schemaVersion	mixed	One row. deviceId is random, used only for the LLM rate limit.

Sharing with friends

Friends get a link, not an install file; their data never reaches you, and the LLM can't cost you money because it's capped.

Getting it on their phone

- Android (Chrome): open the link, tap the install prompt or menu → Install app.
- iPhone: **Safari** → Share → Add to Home Screen. Safari is the most reliable route.
- Put a 3-step install guide on the first screen, with pictures for each phone type.

Privacy you can promise honestly

- Say it on the first screen: "Your data stays on this phone. The developer can't see it."
- Only uncategorised entry text goes to your proxy and on to the LLM provider. Say that too.
- The proxy must not log request bodies. Log only counts.

Cost control

- Per-device cap, e.g. 30 LLM calls a day (deviceId + a counter in Cloudflare KV or similar).
- A global monthly cap. When hit, the app quietly falls back to "pick a category".

- Lock the proxy's CORS to your app's domain so other sites can't use it.
- For 5–20 friends this should cost nothing.

Updates without breaking anything

- Each deploy reaches everyone; show a "New version — tap to reload" toast.
- Change the database shape only through Dexie version migrations. Test a migration on a copy of real exported data first.

Feedback: a "Report a problem" button that opens WhatsApp or a Google Form with the app version pre-filled.

Build phases

About 3–4 weeks of evenings to a version friends can use. Each phase ends on a "done when" test, not on "code written".

1. **Parser first (1–2 evenings).** Write the parser as pure functions with no UI, plus 30+ Vitest cases from real inputs.
   - Done when: every row in the input table above passes, and CI runs the tests on push.
2. **Usable for you (1 weekend).** Quick-add box, confirmation chip, entry list, edit/delete/undo, default categories, Dexie storage, PWA install, deploy to Vercel.
   - Done when: you've logged every expense for 7 straight days on your own phone.
3. **Summaries and backup (3–4 evenings).** Summary screen, CSV/JSON export and import, `navigator.storage.persist()`, backup reminder.
   - Done when: you can answer "where did this month go?" in 10 seconds, and a restore from backup works on a fresh browser.
4. **Smart layer (3–4 evenings).** Keyword memory with the seed list, then the LLM proxy with caps.
   - Done when: fewer than 1 in 10 entries needs a manual category pick (check the `source` field).
5. **Friends beta (2 weeks of use).** Ship to 3–5 friends with the install guide and feedback button.
   - Done when: at least 3 are still logging after 2 weeks. If not, remove friction before adding any feature.
6. **v2 by demand only.** Budgets, reminders, recurring, voice, monthly insight — build only what friends actually ask for, most-requested first.

Rule: don't start phase 5 until you've used it daily for 2 weeks yourself. If you won't use it, they won't either.

Risks and failure modes

The two most likely ways this fails are you stopping and someone losing their data; both are design problems, not code problems.

Risk	Likelihood	What to do
You stop logging after a week or two	High	Entry under 5 s, evening reminder, a lump-sum "untracked" entry so a missed day isn't a reason to quit.
Scope creep delays v1	High	v2 features only after the friends beta, only on request.
Data lost: phone lost, browser data cleared, or iPhone storage cleanup of unused sites	Medium	Install as a home-screen app, call `navigator.storage.persist()`, nag for a backup every 30 days, one-tap export to Drive or WhatsApp.
Parser reads the wrong number ("2 coffees 300")	Medium	Always show the confirmation chip; the test set grows with every bug found.
LLM picks the wrong category	Medium	It's a suggestion chip; corrections feed keyword memory, so the same mistake doesn't repeat.
iPhone gaps (notifications need the app installed; voice input unreliable)	Medium	Treat reminders and voice as Android-first extras.
Someone wants two devices	Low–medium	Export/import first. Real sync only if asked repeatedly — then Supabase with row-level security, which you've used before.
API key leaked or quota drained	Low	Key only in the proxy, per-device and monthly caps, CORS locked to your domain.

Next steps

Start with real data, not code: your own typing habits define the parser.

- [ ] Open your UPI history for last week and write 30 entries the way you'd actually type them. This is your parser test set.
- [ ] Scaffold the project and deploy an empty shell to Vercel (commands below).
- [ ] Build the parser as pure functions until all 30 tests pass.
- [ ] Use it yourself for 7 days before showing anyone.

Scaffold from PowerShell (Windows):

```
npm create vite@latest expense-app -- --template react
cd expense-app
npm install
npm install dexie
npm install -D vite-plugin-pwa vitest
npm run dev
```

- `npm create vite@latest expense-app -- --template react` — creates a new React project folder named expense-app. The `--` passes the template option through npm to Vite.
- `cd expense-app` — moves into that folder.
- `npm install` — downloads the packages the template lists.
- `npm install dexie` — adds the IndexedDB library the app needs at runtime.
- `npm install -D vite-plugin-pwa vitest` — `-D` marks them as dev tools: the PWA build plugin and the test runner. They aren't shipped to the phone as app code.
- `npm run dev` — starts a local server. To test on your phone over Wi-Fi, run `npm run dev -- --host` and open the Network URL it prints.